

```
import numpy as np
import pandas as pd

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# 1. Load PhiUSIIL Dataset

dataset = fetch_ucirepo(id=967)

X = dataset.data.features
y = dataset.data.targets

# 2. Convert Target to Series
```

```
y = y.iloc[:, 0]
```

```
# -----
```

```
# 3. Remove Unnecessary Columns
```

```
# -----
```

```
remove_columns = ["FILENAME"]
```

```
for column in remove_columns:
```

```
    if column in X.columns:
```

```
        X = X.drop(columns=[column])
```

```
# -----
```

```
# 4. Remove URL and Other Text Columns
```

```
# -----
```

```
categorical_columns = X.select_dtypes(
```

```
    include=["object"]
```

```
).columns
```

```
X = X.drop(columns=categorical_columns)
```

```
# -----
```

```
# 5. Convert Features to Numeric
```

```
# -----
```

```
X = X.apply(
```

```
pd.to_numeric,  
errors="coerce"  
)
```

```
X = X.fillna(0)
```

```
# -----
```

```
# 6. Train-Test Split
```

```
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

```
# -----
```

```
# 7. Feature Scaling
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# -----
```

```
# 8. Build Deep Neural Network
```

```
# -----
```

```
model = Sequential([
```

```
    Dense(
```

```
        128,
```

```
        activation="relu",
```

```
        input_shape=(X_train.shape[1],)
```

```
    ),
```

```
    Dropout(0.30),
```

```
    Dense(
```

```
        64,
```

```
        activation="relu"
```

```
    ),
```

```
    Dropout(0.20),
```

```
    Dense(
```

```
        32,
```

```
        activation="relu"
```

```
    ),
```

```
    Dense(
```

```
        1,
```

```
        activation="sigmoid"
    )
])

# -----
# 9. Compile Model
# -----

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# -----
# 10. Display Model
# -----

model.summary()

# -----
# 11. Train Model
# -----

history = model.fit(
    X_train,
    y_train,
    epochs=10,
```

```
    batch_size=256,
    validation_split=0.2
)

# -----
# 12. Predict Test Data
# -----

probability = model.predict(X_test)

y_pred = (
    probability >= 0.5
).astype(int)

# -----
# 13. Calculate Performance
# -----

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred
)
```

```
recall = recall_score(  
    y_test,  
    y_pred  
)
```

```
f1 = f1_score(  
    y_test,  
    y_pred  
)
```

```
# -----  
# 14. Display Results  
# -----
```

```
print("\nMODEL PERFORMANCE")
```

```
print("Accuracy :", accuracy)
```

```
print("Precision :", precision)
```

```
print("Recall   :", recall)
```

```
print("F1 Score :", f1)
```

```
(shared-ai-venv) PS E:\Ruby-Projects\Sentiment_Analysis_LSTM> python .\hi.py
E:\Ruby-Projects\Sentiment_Analysis_LSTM> python .\hi.py:90: PandasWarning: For backward compatibility, str dtypes are included by select_dtypes when object dtype is specified. This behavior is deprecated and will be
future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
categorical_columns = X.select_dtypes(
E:\Ruby-Projects\shared-ai-venv\Lib\site-packages\keras\src\layers\core\dense.py:103: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'In
ject as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790151252.310822 14640 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 120)	6,528
dropout (Dropout)	(None, 120)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

```
Total params: 16,897 (66.00 KB)
Trainable params: 16,897 (66.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
590/590 4s 5ms/step - accuracy: 0.9943 - loss: 0.0206 - val_accuracy: 0.9997 - val_loss: 0.0011
Epoch 2/10
590/590 2s 3ms/step - accuracy: 0.9998 - loss: 8.7194e-04 - val_accuracy: 0.9998 - val_loss: 8.0784e-04
Epoch 3/10
590/590 2s 3ms/step - accuracy: 0.9998 - loss: 6.8322e-04 - val_accuracy: 0.9998 - val_loss: 7.7037e-04
Epoch 4/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 5.9832e-04 - val_accuracy: 0.9999 - val_loss: 4.7508e-04
Epoch 5/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 2.9112e-04 - val_accuracy: 0.9999 - val_loss: 5.4816e-04
Epoch 6/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 3.5383e-04 - val_accuracy: 0.9999 - val_loss: 5.5847e-04
Epoch 7/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 2.3319e-04 - val_accuracy: 0.9999 - val_loss: 7.0695e-04
Epoch 8/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 2.3442e-04 - val_accuracy: 0.9999 - val_loss: 9.4249e-04
Epoch 9/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 3.3828e-04 - val_accuracy: 0.9999 - val_loss: 8.5337e-04
Epoch 10/10
590/590 2s 3ms/step - accuracy: 1.0000 - loss: 1.3390e-04 - val_accuracy: 0.9999 - val_loss: 7.9300e-04
1474/1474 1s 889us/step

MODEL PERFORMANCE
Accuracy : 0.9999575902796921
Precision : 0.9999258490286222
Recall : 1.0
F1 Score : 0.9999629231396685
```

```
❖ (shared-ai-venv) PS E:\Ruby-Projects\Sentiment_Analysis_LSTM>
```

```
# MQTT ATTACK CLASSIFICATION - COMPLETE FIXED CODE
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score
)

from sklearn.ensemble import RandomForestClassifier

# 1. LOAD DATASET
file_path = "RT_IOT2022.csv"

df = pd.read_csv(file_path)

print("=" * 70)
```

```

print("DATASET INFORMATION")

print("=" * 70)


print("Dataset shape:", df.shape)

print("\nFirst 5 rows:")

print(df.head())


# =====

# 2. BASIC CLEANING

# =====


# Remove completely empty rows

df = df.dropna(how="all").copy()


# Remove duplicate rows

df = df.drop_duplicates().copy()


print("\nShape after removing empty/duplicate rows:", df.shape)


# =====

# 3. CHECK TARGET COLUMN

# =====


target_column = "Attack_type"


if target_column not in df.columns:

    raise ValueError(

        f"Target column '{target_column}' was not found.\n"

```

```

        f"Available columns are:\n{df.columns.tolist()}"
    )

# Remove rows where target is missing
df = df.dropna(subset=[target_column]).copy()

# =====

# 4. DISPLAY CLASS DISTRIBUTION

# =====

print("\n" + "=" * 70)
print("ATTACK TYPE DISTRIBUTION - BEFORE FILTERING")
print("=" * 70)

class_counts = df[target_column].value_counts()

print(class_counts)

print("\nNumber of classes:", len(class_counts))

print("\nClasses with fewer than 2 samples:")
print(class_counts[class_counts < 2])

# =====

# 5. REMOVE SINGLE-SAMPLE CLASSES

# =====

#

# train_test_split(..., stratify=y) requires every class to have

```

```

# at least 2 samples.

#
# We remove classes having fewer than 2 samples so that
# stratified splitting works correctly.
#
# =====

valid_classes = class_counts[class_counts >= 2].index

df = df[df[target_column].isin(valid_classes)].copy()

print("\n" + "=" * 70)
print("ATTACK TYPE DISTRIBUTION - AFTER REMOVING SINGLETON CLASSES")
print("=" * 70)

print(df[target_column].value_counts())

print("\nDataset shape after filtering:", df.shape)

# =====

# 6. SEPARATE FEATURES AND TARGET

# =====

X = df.drop(columns=[target_column])
y = df[target_column]

# =====

# 7. REMOVE NON-INFORMATIVE COLUMNS

```

```
# =====  
  
#  
  
# 'no' is usually just a row/index number and should not be used  
# as a predictive feature.  
  
#  
  
# 'id' is also removed if it exists and is merely an identifier.  
  
#  
  
# =====
```

```
columns_to_drop = []
```

```
for col in ["no", "id"]:
```

```
    if col in X.columns:
```

```
        columns_to_drop.append(col)
```

```
if columns_to_drop:
```

```
    X = X.drop(columns=columns_to_drop)
```

```
print("\nFeatures after removing identifier columns:")
```

```
print(X.shape)
```

```
print("\nFeature columns:")
```

```
print(X.columns.tolist())
```

```
# =====  
  
# 8. HANDLE INFINITE VALUES  
  
# =====
```

```
X = X.replace([np.inf, -np.inf], np.nan)
```

```
# =====
```

```
# 9. IDENTIFY NUMERICAL AND CATEGORICAL FEATURES
```

```
# =====
```

```
numeric_features = X.select_dtypes(  
    include=["int64", "int32", "float64", "float32"]  
)  
.columns.tolist()
```

```
categorical_features = X.select_dtypes(  
    include=["object", "category", "bool"]  
)  
.columns.tolist()
```

```
print("\n" + "=" * 70)  
print("FEATURE TYPES")  
print("=" * 70)
```

```
print("Numerical features:", len(numeric_features))  
print("Categorical features:", len(categorical_features))
```

```
print("\nCategorical columns:")  
print(categorical_features)
```

```
# =====
```

```
# 10. ENCODE TARGET LABELS
```

```
# =====
```

```

label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)

print("\n" + "=" * 70)
print("TARGET LABELS")
print("=" * 70)

for number, label in enumerate(label_encoder.classes_):
    print(number, "->", label)

# =====

# 11. TRAIN / TEST SPLIT
# =====

#
# The important fix is:
#
# stratify=y_encoded
#
# Because singleton classes were removed above, every class has
# at least two samples.
#
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.20,

```

```
    random_state=42,  
    stratify=y_encoded  
)
```

```
print("\n" + "=" * 70)  
print("TRAIN / TEST SPLIT")  
print("=" * 70)
```

```
print("X_train:", X_train.shape)  
print("X_test :", X_test.shape)
```

```
print("y_train:", y_train.shape)  
print("y_test :", y_test.shape)
```

```
# =====  
# 12. PREPROCESSING  
# =====
```

```
numeric_transformer = Pipeline(  
    steps=[  
        ("imputer", SimpleImputer(strategy="median")),  
        ("scaler", StandardScaler())  
    ]  
)
```

```
categorical_transformer = Pipeline(  
    steps=[  
        ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
    ("encoder", __import__("sklearn").preprocessing.OneHotEncoder(
        handle_unknown="ignore"
    ))
]
)
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ],
    remainder="drop"
)
```

```
# =====
# 13. RANDOM FOREST MODEL
# =====
```

```
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced"
)
```

```
# =====
# 14. CREATE COMPLETE PIPELINE
# =====
```

```

pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", model)
    ]
)

# =====

# 15. TRAIN MODEL

# =====

print("\n" + "=" * 70)
print("TRAINING RANDOM FOREST")
print("=" * 70)

pipeline.fit(X_train, y_train)

print("Training completed successfully!")

# =====

# 16. PREDICTION

# =====

y_pred = pipeline.predict(X_test)

# =====

# 17. ACCURACY

```

```
# =====
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\n" + "=" * 70)
```

```
print("MODEL ACCURACY")
```

```
print("=" * 70)
```

```
print(f"Accuracy: {accuracy:.4f}")
```

```
print(f"Accuracy: {accuracy * 100:.2f}%")
```

```
# =====
```

```
# 18. CLASSIFICATION REPORT
```

```
# =====
```

```
print("\n" + "=" * 70)
```

```
print("CLASSIFICATION REPORT")
```

```
print("=" * 70)
```

```
print(
    classification_report(
        y_test,
        y_pred,
        target_names=label_encoder.classes_,
        zero_division=0
    )
)
```

```

# =====

# 19. CONFUSION MATRIX

# =====

cm = confusion_matrix(y_test, y_pred)

print("\n" + "=" * 70)
print("CONFUSION MATRIX")
print("=" * 70)

print(cm)

# =====

# 20. SHOW CLASS-WISE TEST DISTRIBUTION

# =====

print("\n" + "=" * 70)
print("TEST SET CLASS DISTRIBUTION")
print("=" * 70)

test_class_counts = pd.Series(y_test).value_counts().sort_index()

for class_number, count in test_class_counts.items():
    print(
        f"{class_number:3d} -> "
        f"{label_encoder.classes_[class_number]:30s} : {count}"
    )

```

```

# =====

# 21. FEATURE IMPORTANCE

# =====

#

# Random Forest feature importance after preprocessing.

#

# =====

try:

    feature_names = pipeline.named_steps[
        "preprocessor"
    ].get_feature_names_out()

    importances = pipeline.named_steps[
        "classifier"
    ].feature_importances_

    feature_importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Importance": importances
    })

    feature_importance_df = feature_importance_df.sort_values(
        by="Importance",
        ascending=False
    )

```

```

print("\n" + "=" * 70)

print("TOP 20 IMPORTANT FEATURES")

print("=" * 70)


print(feature_importance_df.head(20).to_string(index=False))


except Exception as e:

    print("\nCould not calculate feature importance:")
    print(e)


# =====

# 22. FINAL SUMMARY

# =====


print("\n" + "=" * 70)

print("FINAL SUMMARY")

print("=" * 70)


print("Original dataset shape   :", df.shape)
print("Training samples         :", len(X_train))
print("Testing samples           :", len(X_test))
print("Number of features         :", X.shape[1])
print("Number of attack classes   :", len(label_encoder.classes_))
print(f"Test accuracy               : {accuracy * 100:.2f}%")


print("\nAttack classes:")

```

```
for i, label in enumerate(label_encoder.classes_):
```

```
    print(f"{i}: {label}")
```

```
print("\nDone!")
```

```
=====
MODEL ACCURACY
=====
```

```
Accuracy: 0.9974
```

```
Accuracy: 99.74%
```

```
=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
ARP_poisoning	0.99	0.99	0.99	1550
DDOS_Slowloris	1.00	1.00	1.00	107
DOS_SYN_Hping	1.00	1.00	1.00	6726
MQTT_Publish	1.00	1.00	1.00	829
Thing_Speak	0.99	0.99	0.99	1622
Wipro_bulb	0.98	0.94	0.96	51
accuracy			1.00	10885
macro avg	0.99	0.99	0.99	10885
weighted avg	1.00	1.00	1.00	10885

```
=====
CONFUSION MATRIX
=====
```

```
[[1538    0    0    0   12    0]
 [    0  107    0    0    0    0]
 [    0    0 6726    0    0    0]
 [    1    0    0  828    0    0]
 [   11    0    0    0 1610    1]
 [    1    0    0    0    2   48]]
```

